

hw3

May 31, 2026

1 HW3: Hidden Markov Models

STATS305B, Stanford University, Winter 2025

Your name: Zane Liao

Collaborators: DeepSeek V4

Hours spent: 13h30m

This assignment will focus on latent variable models for spatiotemporal data. Here, our focus is on climate data, analyzing long-term trends in global temperature.

- **Data:** Mean temperature in the lower troposphere from [NOAA](#). The data is sampled monthly from 1981 to 2024, and it has global coverage on a $2.5^\circ \times 2.5^\circ$ grid.
- **Model:** We will use **Hidden Markov Models (HMMs)** to look for anomalies in global temperature trends.
- **Algorithm:** We'll fit the model using **Expectation-Maximization (EM)** and the **forward-backward algorithm**.

This assignment will focus on latent variable models for spatiotemporal data. Here, our focus is on climate data, analyzing long-term trends in global temperature.

- **Data:** Mean temperature in the lower troposphere from [NOAA](#). The data is sampled monthly from 1981 to 2024, and it has global coverage on a $2.5^\circ \times 2.5^\circ$ grid.
- **Model:** We will use **Hidden Markov Models (HMMs)** to look for anomalies in global temperature trends.
- **Algorithm:** We'll fit the model using **Expectation-Maximization (EM)** and the **forward-backward algorithm**.

1.1 Setup

```
[4]: %%capture
      %pip install jaxtyping
      %pip install basemap
```

```
[1]: import matplotlib.pyplot as plt
      import numpy as np
```

```

import seaborn as sns
import torch
import torch.nn as nn

from datetime import datetime
from jaxtyping import Float, Bool
from torch import Tensor
from torch.distributions import Normal
from tqdm.auto import trange
from typing import Optional, Tuple

```

1.1.1 Load the data

This dataset was downloaded from [NOAA](#). It consists of the following fields: - **temperature**: a Tensor of shape $T \times D_{\text{lat}} \times D_{\text{lon}}$ where - T denotes the number of timesteps, - D_{lat} specifies the number of latitudes grid points, and - D_{lon} specifies the number of longitude grid points.

Note that the temperature is measured in [degrees Kelvin](#) and it represents the average temperature in the [lower troposphere](#), which extends up to 6-20km from the Earth's surface. The reported temperature is the mean temperature for the corresponding month.

- **latitude**: Tensor of shape $(D_{\text{lat}} + 1) \times (D_{\text{lon}} + 1)$ with a latitude for the edges of each bin in the grid
- **longitude**: Tensor of shape $(D_{\text{lat}} + 1) \times (D_{\text{lon}} + 1)$ with a longitude for the edges of each bin in the grid
- **dates**: Numpy array of shape T and dtype `datetime64` containing the date of each measurement.

We performed some light preprocessing to make the data easier to work with. Specifically, we:

- Extracted the data from the original netCDF4 format and converted it into PyTorch tensors.
- Dropped missing data for the poles ($|\text{latitude}| > 82.5$).
- Created a meshgrid of latitude and longitude for plotting.
- Created a numpy array of dates for the time series.

```

[ ]: # Download the data
!wget -nc https://github.com/slinderman/stats305b/raw/refs/heads/winter2025/
↪assignments/hw3/hw3.pt

```

File 'hw3.pt' already there; not retrieving.

```

[2]: hw3_dataset = torch.load('hw3.pt', weights_only=False)
temperature = hw3_dataset['temperature']
latitude = hw3_dataset['latitude']
longitude = hw3_dataset['longitude']
dates = hw3_dataset['dates']

```

1.1.2 Helper function for plotting

```
[3]: def plot_temperature(data : Float[Tensor, "n_latitude n_longitude"],
    lats : Float[Tensor, "n_latitude n_longitude"],
    lons : Float[Tensor, "n_latitude n_longitude"],
    title : Optional[str]=None,
    cmap : Optional[np.array]=sns.color_palette("icefire", 256),
    ↪as_cmap=True),
    ax: Optional[plt.Axes]=None,
    **kwargs) -> None:

    """
    Plot the temperature data using Basemap.

    Parameters
    -----
    data : Float[Tensor, "n_latitude n_longitude"]
        The temperature data to plot.

    lats : Float[Tensor, "n_latitude n_longitude"]
        A meshgrid of latitudes

    lons : Float[Tensor, "n_latitude n_longitude"]
        A meshgrid of longitudes

    title : Optional[str]
        The title of the plot

    cmap : Optional[np.Array]
        The colormap to use for the plot

    ax : Optional[plt.Axes]
        The axes to plot on

    **kwargs:
        Additional arguments to pass to the Basemap `pcolormesh` function.
        For example, you may want to set the `vmin` and `vmax` values for the
    ↪colormap.
    """
    try:
        from mpl_toolkits.basemap import Basemap
    except:
        print("Basemap is not installed, but it is not strictly required for
    ↪this assignment. " \
            "If you'd like to generate this plot, try running `conda install
    ↪basemap` " \
            " or `pip install basemap`. Note that basemap is not currently
    ↪working in "\
```

```

        "Python 3.13.")
    return

    # create figure, axes instances.
    if ax is None:
        fig = plt.figure()
        ax = fig.add_axes([0.05,0.05,0.9,0.9])

    # create Basemap instance.
    m = Basemap(projection='mill',
                llcrnrlon=-180,
                llcrnrlat=-82.5,
                urcrnrlon=180,
                urcrnrlat=82.5,
                ax=ax)

    # draw boundarie and coasts
    m.drawmapboundary(fill_color='0.8')
    m.drawcoastlines(linewidth=1)

    # plot temperature as a colormap
    im = m.pcolormesh(np.array(lons),
                    np.array(lats),
                    data,
                    shading='flat',
                    cmap=cmap,
                    latlon=True,
                    **kwargs)

    # draw parallels and meridians, but don't bother labelling them.
    m.drawparallels(np.arange(-90.,90.,30.))
    m.drawmeridians(np.arange(-180.,180.,60.))
    m.colorbar(im,"bottom", size="5%", pad="2%", label="°K")
    ax.set_title(title)

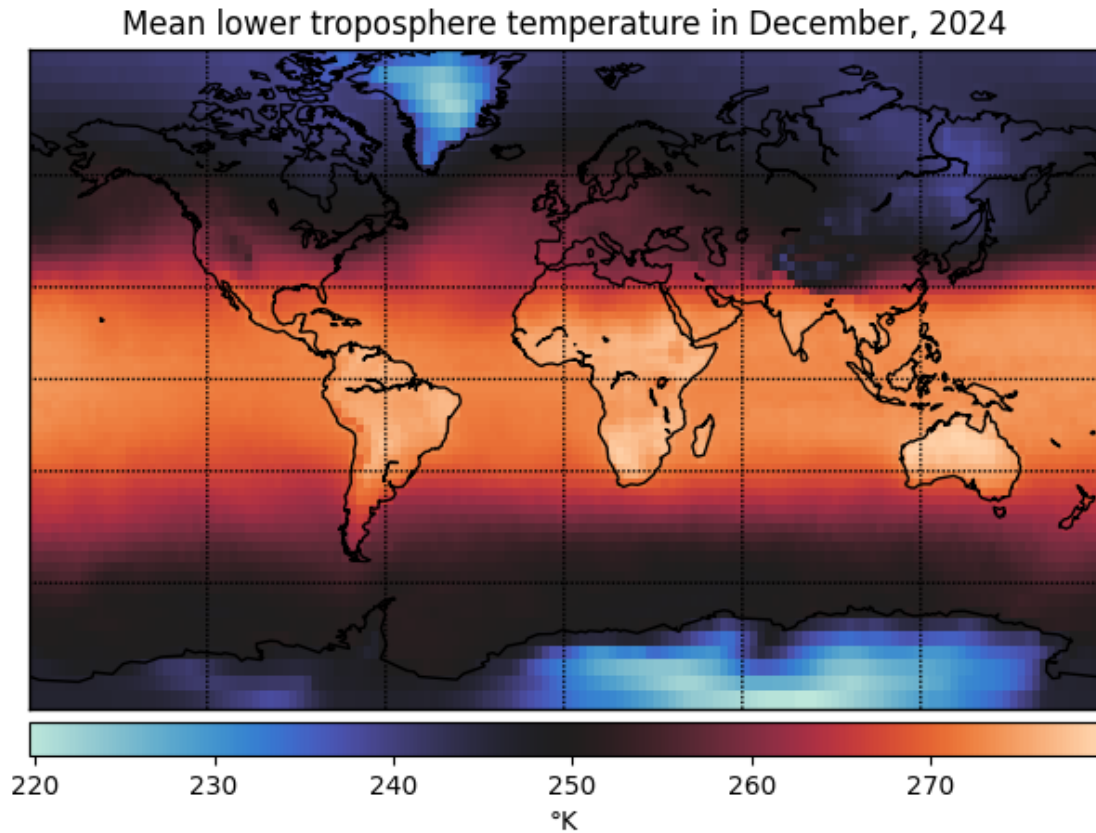
```

You can use this function to visualize the global temperature as a heatmap.

```

[ ]: t = -1
date_to_str = lambda date: date.astype(datetime).strftime('%B, %Y')
plot_temperature(temperature[t], latitude, longitude,
                title=f"Mean lower troposphere temperature in_
↪ {date_to_str(dates[t])}")

```

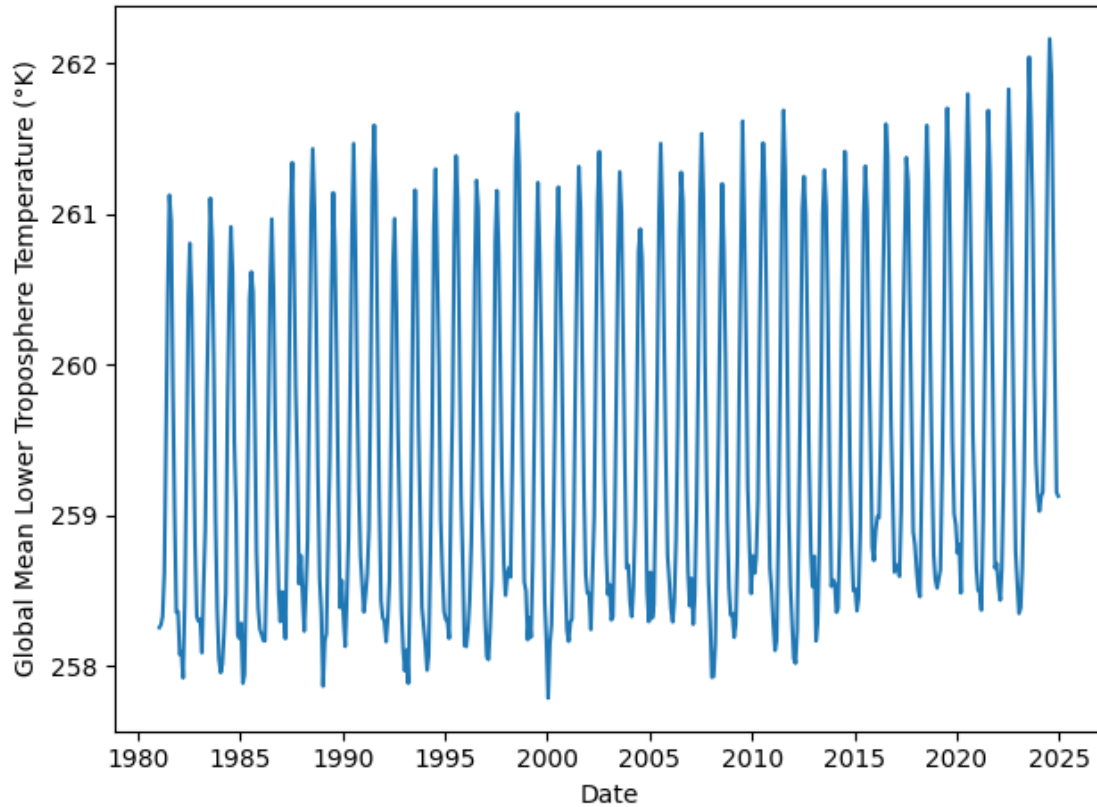


1.1.3 Extract global mean temperature over time

For the first part of this assignment, we will ignore the spatial component of the data and focus on the time series of global mean temperatures.

```
[4]: global_temps = temperature.mean(axis=(1,2))

plt.plot(dates, global_temps)
plt.xlabel('Date')
plt.ylabel('Global Mean Lower Troposphere Temperature (°K)')
plt.tight_layout()
```



```
[5]: temperature.shape, latitude.shape, longitude.shape, dates.shape
```

```
[5]: (torch.Size([528, 66, 144]),
      torch.Size([67, 145]),
      torch.Size([67, 145]),
      (528,))
```

You can see the trend by eye: Over the past 45 years, the global mean temperature in the lower troposphere has been rising by about 0.14 °K each decade. (The Kelvin scale is a shifted version of the Celcius scale, so this is equivalent to 0.14 °C/decade.)

In this assignment, we will model these trends and look for anomalies using hidden Markov models (HMM).

1.2 Part 0: Establishing a Baseline

Our first step is to establish a baseline using linear regression. Let, - $T = 528$ denote the number of months of measurements in the dataset. - $y_t \in \mathbb{R}$ denote the global mean temperature in month $t = 1, \dots, T$. - $d_t \in \{1, \dots, 12\}$ denote which month the measurement was made in.

1.2.1 Problem 0a: Fit a linear model

Compute $\hat{\beta}_{\text{MLE}}$, the maximum likelihood estimate of the weights for a linear regression model,

$$y_t = \beta^\top \mathbf{x}_t + \epsilon_t$$

where are assumed to be ϵ_t are independent, mean zero noise terms.

Use the following covariates,

$$\mathbf{x}_t = (t, \mathbb{I}[d_t = 1], \dots, \mathbb{I}[d_t = 12]) \in \mathbb{R}^{13}.$$

Answer(cs229notes-1.2.2):

$$\hat{\beta}_{\text{MLE}} = (X^T X)^{-1} X^T y$$

```
[6]: # We'll help you extract the months as integers
months = torch.tensor([date.astype(datetime).month for date in dates])
T = len(months)
t = torch.arange(1, T+1, dtype=torch.float32)
X = torch.cat([t.unsqueeze(1), nn.functional.one_hot(months - 1,
    ↪ num_classes=12)], dim=1).float()
y = global_temps.float()

beta_hat_mle = (X.T @ X).inverse() @ X.T @ y
beta_hat_mle
# YOUR CODE HERE
```

```
[6]: tensor([1.2459e-03, 2.5805e+02, 2.5799e+02, 2.5806e+02, 2.5859e+02, 2.5951e+02,
          2.6055e+02, 2.6103e+02, 2.6066e+02, 2.5980e+02, 2.5892e+02, 2.5831e+02,
          2.5814e+02])
```

1.2.2 Problem 0b: Plot the predictions and residuals

Plot the measurements y predictions and $\hat{y}_t = \hat{\beta}_{\text{MLE}}^\top \mathbf{x}_t$.

In a separate panel, plot the residuals $r_t \triangleq y_t - \hat{y}_t$.

Remember to label your axes!

```
[7]: # YOUR CODE HERE
y_hat_t = X @ beta_hat_mle
r_t = y - y_hat_t

fig, axs = plt.subplots(1, 2, figsize=(13, 5))
axs[0].plot(dates, y, label="Measurements dataset")
axs[0].plot(dates, y_hat_t, label="Predictions dataset")
axs[0].set_xlabel("Date")
axs[0].set_ylabel("Global Mean Lower Troposphere Temperature (°K)")
axs[0].legend()
```

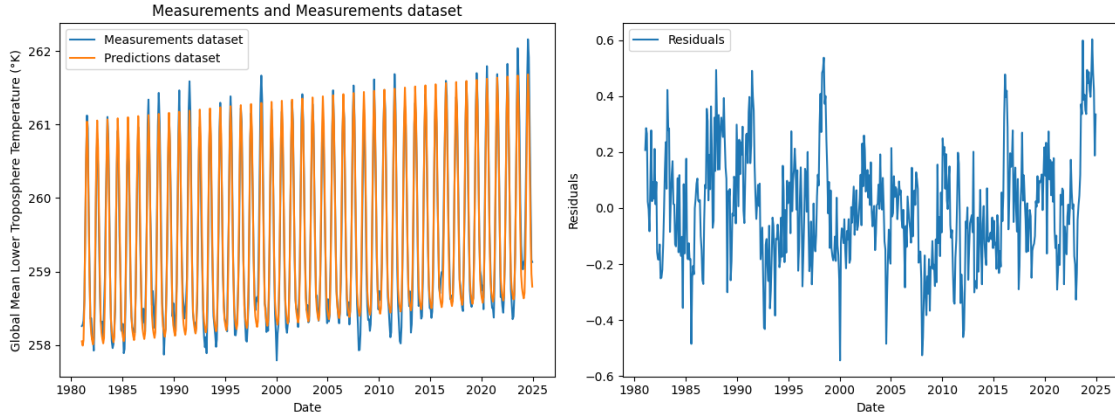
```

axs[0].set_title("Measurements and Measurements dataset")

axs[1].plot(dates, r_t, label="Residuals")
axs[1].set_xlabel("Date")
axs[1].set_ylabel("Residuals")
axs[1].legend()

plt.tight_layout()

```



1.2.3 Problem 0c: Discuss

How do you interpret the estimated coefficients of this model?

Answer:

We estimated the model coefficients using the least squares method. The coefficients of the time covariate (Linear Trend) quantify the average rate of global warming over the past 44 years, while the monthly indices reconstruct the average climatology of global temperature.

1.3 Part 1: State Inference with the Forward-Backward Algorithm

You should see that the residuals look autocorrelated: if r_t is positive, r_{t+1} is likely to be positive as well. There are several factors that could lead to such autocorrelation; one is the [El Niño Southern Oscillation \(ENSO\)](#). These global variations lead to periods of warming (El Niño) and cooling (La Niña) that last around a year and occur every 2-7 years (according to Wikipedia). We will attempt to infer these events using a Hidden Markov Model (HMM).

Let $z_t \in \{1, \dots, K\}$ denote a **discrete latent state** at time t . We will model the residuals as *conditionally independent* given the latent states,

$$r_t \mid z_t = k \sim N(\mu_k, \sigma_k^2),$$

where (μ_k, σ_k^2) denote the mean and variance of the residuals in state k .

We model the latent states as a Markov process,

$$\Pr(z_{t+1} = j \mid z_t = i) = p_{ij},$$

where p_{ij} are the elements of the transition matrix,

$$\mathbf{P} = \begin{bmatrix} p_{11} & \cdots & p_{1K} \\ \vdots & & \vdots \\ p_{K1} & \cdots & p_{KK} \end{bmatrix}$$

Finally, we assume

$$z_1 \sim \text{Cat}(\pi_0)$$

where π_0 is the initial distribution.

The parameters of the model are $\theta = (\{\mu_k, \sigma_k^2\}_{k=1}^K, \mathbf{P}, \pi_0)$. Our goal is to estimate the parameters and infer the posterior marginal distributions of the latent states.

1.3.1 Problem 1a: Forward-Backward Algorithm

Implement the forward-backward algorithm for computing posterior marginal probabilities,

$$\Pr(z_t = k \mid r_{1:T}; \theta) = \mathbb{E}[\mathbb{I}[z_t = k] \mid r_{1:T}; \theta].$$

Also, compute the log marginal likelihood, $\log p(r_{1:T}; \theta)$, during the forward pass.

```
[8]: def forward_pass(initial_probs: Float[Tensor, "num_states"],
    transition_matrix: Float[Tensor, "num_states num_states"],
    log_likes: Float[Tensor, "num_timesteps num_states"]) \
    -> Tuple[Float[Tensor, "num_timesteps num_states"], Float]:

    r"""
    Compute the filtering probabilities,  $p(z_t \mid r_{1:t-1})$  and the marginal
    log likelihood,  $\log p(r_{1:T})$ , where  $z_t$  is the hidden state at time  $t$ 
    and  $r_{1:t-1}$  is the observed data up to time  $t-1$ .

    Parameters
    -----
    initial_probs : Float[Tensor, "num_states"]
        The initial state probabilities,  $p(z_1)$ .

    transition_matrix : Float[Tensor, "num_states num_states"]
        The state transition probabilities,  $p(z_t \mid z_{t-1})$ .

    log_likes : Float[Tensor, "num_timesteps num_states"]
        The log likelihoods,  $\log p(r_t \mid z_t)$ .

    Returns
    -----
```

```

    alphas : Float[Tensor, "num_timesteps num_states"]
        The filtering probabilities,  $p(z_t | r_{1:t-1})$ .

    marginal_loglike : Float
        The marginal log likelihood,  $\log p(r_{1:T})$ .

    """
    # YOUR CODE HERE
    T, k = log_likes.shape

    alphas = torch.zeros((T, k), device=log_likes.device)
    marginal_loglike = torch.tensor(0.0, device=log_likes.device)

    curr_alpha = initial_probs

    for t in range(T):
        alphas[t] = curr_alpha

        log_joint_t = torch.log(curr_alpha + 1e-12) + log_likes[t]

        log_c_t = torch.logsumexp(log_joint_t, dim=0)

        marginal_loglike += log_c_t

        if t < T - 1:
            filtering_prob = torch.exp(log_joint_t - log_c_t)

            curr_alpha = filtering_prob @ transition_matrix

    return alphas, marginal_loglike

def backward_pass(transition_matrix: Float[Tensor, "num_states num_states"],
                  log_likes: Float[Tensor, "num_timesteps num_states"]) \
    -> Float[Tensor, "num_timesteps num_states"]:
    r"""
    Compute the backward probabilities,  $\beta_t \propto p(z_t, r_{t+1:T})$ 

    Parameters
    -----
    transition_matrix : Float[Tensor, "num_states num_states"]
        The state transition probabilities,  $p(z_t | z_{t-1})$ .

    log_likes : Float[Tensor, "num_timesteps num_states"]
        The log likelihoods,  $\log p(r_t | z_t)$ .

    Returns
    """

```

```

-----
betas : Float[Tensor, "num_timesteps num_states"]
    The backward probabilities,  $p(z_t, r_{t+1:T})$ .

"""
# YOUR CODE HERE
T, k = log_likes.shape

betas = torch.zeros((T, k), device=log_likes.device)

curr_beta = torch.ones(k, device=log_likes.device)

for t in range(T - 1, -1, -1):
    betas[t] = curr_beta

    if t > 0:
        prev_beta = transition_matrix @ (torch.exp(log_likes[t]) *
↪ curr_beta)

        curr_beta = prev_beta / (prev_beta.sum() + 1e-12)

return betas

def forward_backward(initial_probs: Float[Tensor, "num_states"],
                    transition_matrix: Float[Tensor, "num_states num_states"],
                    log_likes: Float[Tensor, "num_timesteps num_states"]) \
    -> Tuple[Float[Tensor, "num_timesteps num_states"], Float]:
    r""" Run the forward backward algorithm to compute the posterior expected_
↪ states,
     $\Pr(z_t=k \mid r_{1:T})$  for all  $t$  and  $k$ , and the marginal log likelihood,
     $\log p(r_{1:T})$ .

    Parameters
    -----
    initial_probs : Float[Tensor, "num_states"]
        The initial state probabilities,  $p(z_1)$ .

    transition_matrix : Float[Tensor, "num_states num_states"]
        The state transition probabilities,  $p(z_t \mid z_{t-1})$ .

    log_likes : Float[Tensor, "num_timesteps num_states"]
        The log likelihoods,  $\log p(r_t \mid z_t)$ .

    Returns
    -----
    expected_states : Float[Tensor, "num_timesteps num_states"]

```

```

        The expected states,  $\Pr(z_t=k \mid r_{1:t-1})$  for all  $t$  and  $k$ .

    marginal_loglike : Float
        The marginal log likelihood,  $\log p(r_{1:T})$ .
    """
    # YOUR CODE HERE
    alphas, marginal_loglike = forward_pass(initial_probs, transition_matrix,
    ↪ log_likes)
    betas = backward_pass(transition_matrix, log_likes)

    log_alphas = torch.log(alphas + 1e-12)
    log_betas = torch.log(betas + 1e-12)

    log_sum = log_alphas + log_likes + log_betas

    expected_states = torch.exp(log_sum - torch.logsumexp(log_sum, dim=-1,
    ↪ keepdim=True))

    return expected_states, marginal_loglike

```

Test Case Compare to exact computations for a small model with $K = 2$ states and $T = 3$ timesteps.

Note: This test is necessary but not sufficient to ensure that your code is correct and stable!

```

[9]: def test_1a():
    torch.manual_seed(305 + ord('b'))
    initial_probs = torch.ones(2) / 2
    transition_matrix = torch.eye(2) * 0.9 + (1 - torch.eye(2)) * 0.1
    log_likes = torch.randn(3, 2)

    # Run your code
    expected_states, marginal_loglike = forward_backward(initial_probs,
    ↪ transition_matrix, log_likes)

    # Compute the log joint probs explicitly
    log_joint_probs = torch.zeros(2, 2, 2)
    for z1 in range(2):
        for z2 in range(2):
            for z3 in range(2):
                log_joint_probs[z1, z2, z3] = \
                    torch.log(initial_probs[z1]) + \
                    torch.log(transition_matrix[z1, z2]) + \
                    torch.log(transition_matrix[z2, z3]) + \
                    log_likes[0, z1] + \
                    log_likes[1, z2] + \
                    log_likes[2, z3]

```

```

# Compare the marginal log likelihood
marginal_loglike_explicit = torch.logsumexp(log_joint_probs, dim=(0, 1, 2))
assert torch.allclose(marginal_loglike, marginal_loglike_explicit)

# Compare the posterior marginal probabilities
posterior_probs = torch.exp(log_joint_probs - marginal_loglike_explicit)
assert torch.allclose(expected_states[0], posterior_probs.sum(dim=(1,2)))
assert torch.allclose(expected_states[1], posterior_probs.sum(dim=(0,2)))
assert torch.allclose(expected_states[2], posterior_probs.sum(dim=(0,1)))

test_1a()

```

1.3.2 Problem 1b: Compute the posterior probability of the latent states

To start, let's suppose we manually defined the following states:

1. **Neutral state** in which $\mu_1 = 0$.
2. **El Niño state** in which $\mu_2 = 0.25$.
3. **La Niña state** in which $\mu_3 = -0.25$.

In all three states, assume $\sigma_k^2 = 0.15^2$. These states correspond to the residuals being neutral, larger than average, or smaller than average, respectively.

Moreover, assume the initial distribution is uniform and the transition probabilities are $p_{kk} = \alpha$ for all $k = 1, \dots, K$ and $p_{ij} = \frac{1-\alpha}{K-1}$ for all $i \neq j$. Set $\alpha = \frac{11}{12}$ for this analysis.

You should:

1. **Compute:** the expected states (i.e., posterior marginal probabilities) using your forward-backward implementation.
2. **Print:** the marginal log likelihood of the data under these parameters.
3. **Plot:** the posterior marginal probability of being in the El Niño state over time.

For comparison, we have provided El Niño events as determined by NOAA based on sea surface temperatures. These won't align perfectly your inferences based on global mean lower tropospheric temperature, but they are still a good sanity check. Show these times on your graph as well.

Remember to label your plots!

```

[10]: # El Nino periods from https://origin.cpc.ncep.noaa.gov/products/
      ↪ analysis_monitoring/ensostuff/ONI_v5.php
NOAA_EL_NINO_PERIODS = [
    (datetime(1982, 4, 1), datetime(1983, 6, 1)),
    (datetime(1986, 9, 1), datetime(1988, 2, 1)),
    (datetime(1991, 5, 1), datetime(1992, 6, 1)),
    (datetime(1994, 9, 1), datetime(1995, 3, 1)),
    (datetime(1997, 5, 1), datetime(1998, 5, 1)),
    (datetime(2002, 6, 1), datetime(2003, 2, 1)),
    (datetime(2004, 7, 1), datetime(2005, 2, 1)),

```

```

(datetime(2006, 9, 1), datetime(2007, 1, 1)),
(datetime(2009, 7, 1), datetime(2010, 3, 1)),
(datetime(2014, 10, 1), datetime(2016, 4, 1)),
(datetime(2018, 9, 1), datetime(2019, 6, 1)),
(datetime(2023, 5, 1), datetime(2024, 4, 1)),
]

```

```

[11]: ## YOUR CODE HERE
mu = torch.tensor([0.0, 0.25, -0.25])
K = 3
sigma = 0.15
alpha = 11 / 12

initial_probs = torch.tensor(np.ones(3) / 3, dtype=torch.float32)

P = np.full((K, K), (1 - alpha) / (K - 1))
np.fill_diagonal(P, alpha)
transition_matrix = torch.tensor(P, dtype=torch.float32)

log_likes = Normal(mu.unsqueeze(0), sigma).log_prob(r_t.unsqueeze(1))

expected_states, marginal_loglike = forward_backward(initial_probs,
↳ transition_matrix, log_likes)

expected_states.shape, marginal_loglike

```

```

[11]: (torch.Size([528, 3]), tensor(215.2275))

```

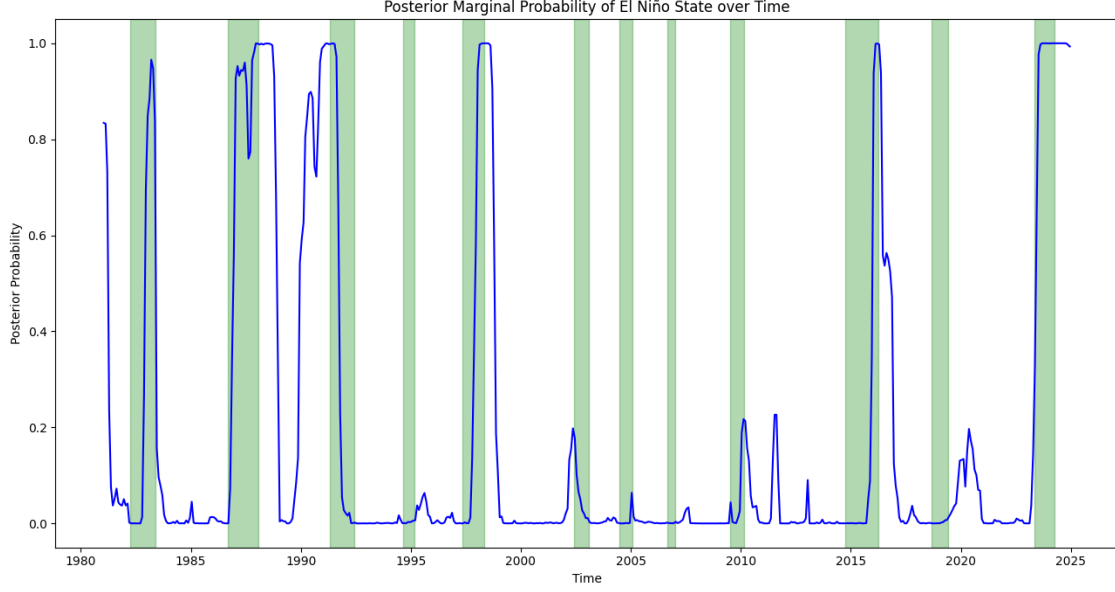
```

[12]: el_nino = expected_states[:, 1]
plt.figure(figsize=(13, 7))
plt.plot(dates, el_nino, label="Posterior P(El Niño state)", color='blue')

for start, end in NOAA_EL_NINO_PERIODS:
    plt.axvspan(start, end, color='green', alpha=0.3, label="NOAA El Niño_
↳ period")

plt.xlabel("Time")
plt.ylabel("Posterior Probability")
plt.title("Posterior Marginal Probability of El Niño State over Time")
plt.tight_layout()
plt.show()

```



1.3.3 Problem 1c: Sensitiity Analysis

Describe how your results change as you increase or decrease the *stay probability*, α .

Answer:

The dwell probability (α) controls the persistence of the state, and thus the autocorrelation of the outcome. When our alpha is less than 0.5, indicating a negative correlation, it means that if this year is an El Niño year, the El Niño phenomenon is unlikely to continue next year. Conversely, if it is greater than 0.5, the El Niño phenomenon is highly likely to continue next year.

1.3.4 Problem 1d: Run Length Analysis

The run length is the duration of a latent state; i.e., the number of time steps the latent state stays in state k before transitioning to a new state. Under the prior with stay probability α , how is the run length distributed? What is its expected value? Explain your findings from Problem 1c in light of the prior distribution of run lengths.

Answer:

We consider it as a Bernoulli distribution, which determines whether the state changes at each step.

$$\text{Given } z_t = i, \quad \Pr(z_{t+1} | z_t = i) = \begin{cases} \alpha & \text{if } z_{t+1} = i \quad (\text{stay}) \\ 1 - \alpha & \text{if } z_{t+1} \neq i \quad (\text{leave}) \end{cases}$$

From the moment the state begins until the first departure, the number of stops L follows a geometric distribution, $L \sim \text{Geom}(1 - \alpha)$. For a geometric distribution of support set starting from 1, the expected value is:

$$\mathbb{E}[L] = \frac{1}{1 - \alpha}$$

- When $\alpha = 0.5$, $\mathbb{E}[L] = 2$, the average stay is 2 steps. - When $\alpha > 0.5$, the expectation is >2 , and the state is more persistent. - When $\alpha < 0.5$, the expectation is <2 , and the state switches quickly.

From the Problem 1c, we found:

1. **The larger α is** (e.g., 0.9) $\rightarrow$ the expected run length is very long (e.g., 10 steps) $\rightarrow$ the state in the time series will last longer, and the autocorrelation is strong.
2. $\alpha = 0.5 \rightarrow$ Expected duration is 2 steps, state transition is relatively fast, but still positive autocorrelation (because $\alpha > 0$).
3. $\alpha < 0.5 \rightarrow$ Expected run length is less than 2, meaning the state will leave in less than 2 steps on average. The chain exhibits **negative autocorrelation**: because once a state is entered, the next step has a higher probability of transitioning to another state, resulting in alternating high and low values.

1.4 Part 2: Parameter Estimation with EM

In Part 1 we just eyeballed the residuals to guess appropriate means and variances. Let's be a bit more rigorous now. Here, we will fix the mean of the neutral state, $\mu_1 = 0$, but we will infer the remaining means and variances $\sigma_1^2, \{\mu_k, \sigma_k^2\}_{k=2}^K$.

We'll still make a few simplifying assumptions: As before, we'll stick with $K = 3$ states, a uniform initial distribution, and the same transition matrix as above with $\alpha = \frac{11}{12}$.

1.4.1 Problem 2a: Derive the M-step

Derive the M-step for the emission means and variances. Let $\omega_{tk} = \Pr(z_t = k \mid r_{1:T}; \theta')$ denote the posterior marginal probabilities from the forward-backward algorithm.

Answer:

ELBO is:

$$\begin{aligned}
\mathbb{L}(\theta) &= \mathbb{E}_{q(z_{1:T})} \left[\sum_{t=1}^T \sum_{k=1}^K \mathbf{1}[z_t = k] \log p(r_t; \theta_k) \right] \\
&= \sum_{t=1}^T \sum_{k=1}^K \mathbb{E}_{q(z_{1:T})} [\mathbf{1}[z_t = k]] \log p(r_t; \theta_k) \quad (\text{Due to the linearity of expectation, } \mathbb{E}_{q(z_{1:T})} [\mathbf{1}[z_t = k]] = q(z_t = k) =: \omega_{tk}) \\
&= \sum_{t=1}^T \sum_{k=1}^K \omega_{tk} \log p(r_t; \theta_k) \\
\log p(r_t; \mu_k, \sigma_k^2) &= -\frac{1}{2} \log(2\pi\sigma_k^2) - \frac{(r_t - \mu_k)^2}{2\sigma_k^2} \quad (\text{Substituting into the Gaussian density}), \text{ So:} \\
&= -\frac{1}{2} \sum_{t=1}^T \sum_{k=1}^K \omega_{tk} \left[\log(2\pi\sigma_k^2) + \frac{(r_t - \mu_k)^2}{\sigma_k^2} \right]
\end{aligned}$$

Differentiate μ_k (only $k = 2, 3$ are free, $\mu_1 = 0$ are fixed), Consider only terms related to μ_k :

$$\mathcal{L}_{\mu_k} = -\frac{1}{2} \sum_{t=1}^T \omega_{tk} \frac{(r_t - \mu_k)^2}{\sigma_k^2}$$

Note that σ_k^2 is considered a constant in this case (when calculating partial derivatives). Derive μ_k :

$$\frac{\partial \mathcal{L}_{\mu_k}}{\partial \mu_k} = -\frac{1}{2} \sum_{t=1}^T \omega_{tk} \cdot \frac{2(r_t - \mu_k)(-1)}{\sigma_k^2} = \sum_{t=1}^T \omega_{tk} \frac{r_t - \mu_k}{\sigma_k^2}$$

Let it equal 0:

$$\sum_{t=1}^T \omega_{tk}(r_t - \mu_k) = 0 \quad \Rightarrow \quad \mu_k \sum_{t=1}^T \omega_{tk} = \sum_{t=1}^T \omega_{tk} r_t$$

So, we get:

$$\mu_k = \frac{\sum_{t=1}^T \omega_{tk} r_t}{\sum_{t=1}^T \omega_{tk}}, \quad k = 2, 3$$

Differentiate with respect to σ_k^2 (for all $k = 1, 2, 3$), Terms related to σ_k^2 :

$$\mathcal{L}_{\sigma_k^2} = \sum_{t=1}^T \omega_{tk} \left[-\frac{1}{2} \log(2\pi\sigma_k^2) - \frac{(r_t - \mu_k)^2}{2\sigma_k^2} \right]$$

Let $\nu = \sigma_k^2$, differentiate with respect to ν :

$$\begin{aligned} \frac{\partial}{\partial \nu} \left(-\frac{1}{2} \log(2\pi\nu) \right) &= -\frac{1}{2\nu} \\ \frac{\partial}{\partial \nu} \left(-\frac{(r_t - \mu_k)^2}{2\nu} \right) &= \frac{(r_t - \mu_k)^2}{2\nu^2} \end{aligned}$$

So far, we get:

$$\frac{\partial \mathcal{L}_{\sigma_k^2}}{\partial \nu} = \sum_{t=1}^T \omega_{tk} \left(-\frac{1}{2\nu} + \frac{(r_t - \mu_k)^2}{2\nu^2} \right)$$

Let it equal 0:

$$\sum_{t=1}^T \omega_{tk} \left(-\frac{1}{2\nu} + \frac{(r_t - \mu_k)^2}{2\nu^2} \right) = 0$$

Multiply by $2\nu^2$ ($\nu > 0$):

$$\begin{aligned} \sum_{t=1}^T \omega_{tk} (-\nu + (r_t - \mu_k)^2) &= 0 \\ \sum_{t=1}^T \omega_{tk} (r_t - \mu_k)^2 &= \nu \sum_{t=1}^T \omega_{tk} \end{aligned}$$

So:

$$\sigma_k^2 = \frac{\sum_{t=1}^T \omega_{tk} (r_t - \mu_k)^2}{\sum_{t=1}^T \omega_{tk}}, \quad k = 1, 2, 3$$

Note that: For $k = 1$, substitute ($\mu_1 = 0$).

- **Neutral state** ($k = 1$):

$$\begin{aligned} \mu_1 &= 0 \text{ (fixed)} \\ \sigma_1^2 &= \frac{\sum_{t=1}^T \omega_{t1} r_t^2}{\sum_{t=1}^T \omega_{t1}} \end{aligned}$$

- **La Niña** ($k = 2$) and **El Niño** ($k = 3$):

$$\mu_k = \frac{\sum_{t=1}^T \omega_{tk} r_t}{\sum_{t=1}^T \omega_{tk}}$$

$$\sigma_k^2 = \frac{\sum_{t=1}^T \omega_{tk} (r_t - \mu_k)^2}{\sum_{t=1}^T \omega_{tk}}$$

1.4.2 Problem 2b: Implement a Gaussian HMM

Complete the code below to implement a Gaussian HMM.

Note: This model allows for missing data to be specified by a `mask`. Mathematically, let $m_t \in \{0, 1\}$ denote a mask, with $m_t = 1$ indicating that r_t is observed and $m_t = 0$ that r_t is missing. We can incorporate the mask into the model by adapting the likelihood as follows,

$$p(r_t | z_t = k, m_t; \theta) \propto [\mathcal{N}(r_t | \mu_k, \sigma_k^2)]^{\mathbb{I}[m_t=1]}.$$

Then, when $m_t = 0$, the likelihood is proportional to one. Equivalently, the log likelihood goes to zero for missing timesteps.

```
[ ]: class GaussianHMM:
    """An implementation of a Gaussian Hidden Markov Model and the
    Expectation-Maximization algorithm for fitting the model to data.

    This model assumes that the initial distribution and transition matrix
    are fixed. Moreover, we assume the mean of the first state is 0, as
    described above.
    """
    def __init__(self,
                  num_states : int,
                  initial_probs: Float[Tensor, "num_states"],
                  transition_matrix : Float[Tensor, "num_states num_states"],
                  initial_means : Optional[Float[Tensor, "num_states"]] = None,
                  initial_variances : Optional[Float[Tensor, "num_states"]] =
    ↪None) -> None:
        self.num_states = num_states
        self.initial_probs = initial_probs
        self.transition_matrix = transition_matrix
        assert torch.allclose(transition_matrix.sum(dim=1), torch.
    ↪ones(num_states))

        # Initialize the means and variances
        if initial_means is not None:
            assert initial_means.shape == (num_states,)
            self.means = initial_means
        else:
            self.means = torch.cat([torch.zeros(1), torch.randn(num_states -
    ↪1)])
```

```

if initial_variances is not None:
    assert initial_variances.shape == (num_states,)
    self.variances = initial_variances
else:
    self.variances = 0.15**2 * torch.ones(num_states)

def e_step(self,
            data: Float[Tensor, "num_timesteps"],
            mask: Bool[Tensor, "num_timesteps"]) \
    -> Tuple[Float[Tensor, "num_timesteps num_states"], Float]:
    r"""Perform the E-step of EM using the forward-backward algorithm.

    Parameters
    -----
    data : Float[Tensor, "num_timesteps"]
        The observed data.

    mask : Optional[Bool[Tensor, "num_timesteps"]]
        A boolean mask indicating which data points are observed.

    Returns
    -----
    expected_states : Float[Tensor, "num_timesteps num_states"]
        The expected states,  $\$Pr(z_{t=k} \mid r_{1:T})\$$  for all  $\$t\$$  and  $\$k\$$ 

    marginal_loglike : Float
        The marginal log likelihood,  $\$\log p(r_{1:T})\$$ .
    """
    # YOUR CODE HERE
    log_likelihoods = Normal(self.means.unsqueeze(0), torch.sqrt(self.
    ↪variances.unsqueeze(0))).log_prob(data.unsqueeze(1)) * mask.unsqueeze(1)
    expected_states, marginal_loglike = forward_backward(self.
    ↪initial_probs, self.transition_matrix, log_likelihoods)
    return expected_states, marginal_loglike

def m_step(self,
            data : Float[Tensor, "num_timesteps"],
            mask : Bool[Tensor, "num_timesteps"],
            expected_states: Float[Tensor, "num_timesteps num_states"]) ->
    ↪None:
    r"""Perform the M-step of EM to update the model parameters.

    Note: this function should update the `means` and `variances`
    ↪attributes in place,
        but the first state mean should remain fixed at 0.

```

Note: Be careful to avoid division by zero if, e.g., expected states_ sums to 0.

Parameters

data : Float[Tensor, "num_timesteps"]

The observed data.

mask : Optional[Bool[Tensor, "num_timesteps"]]

A boolean mask indicating which data points are observed.

expected_states : Float[Tensor, "num_timesteps num_states"]

The expected states, $\Pr(z_t=k \mid r_{1:T})$ for all t and k .

"""

YOUR CODE HERE

Note: Remember to fix the mean of the first state to 0

mask_float = mask.float()

for k in range(self.num_states):

if k == 0:

self.means[k] = 0.0

else:

*weight = torch.sum(expected_states[:, k] * mask_float)*

*weighted = torch.sum(expected_states[:, k] * mask_float * data)*

if weight > 0:

self.means[k] = weighted / weight

else:

self.means[k] = self.means[k]

for k in range(self.num_states):

*weighted_diff = torch.sum(expected_states[:, k] * mask_float * (*

*(data - self.means[k])**2)*

*weight = torch.sum(expected_states[:, k] * mask_float)*

if weight > 0:

self.variances[k] = weighted_diff / weight

self.variances[k] = torch.clamp(self.variances[k], min=1e-7)

pass

def fit(self,

data: Float[Tensor, "num_timesteps"],

mask: Optional[Bool[Tensor, "num_timesteps"]] = None,

num_iters: int = 50,

verbose: bool=True):

"""Fit the HMM with EM.

Parameters

```

-----
data : Float[Tensor, "num_timesteps"]
      The observed data.

mask : Optional[Bool[Tensor, "num_timesteps"]]
      A boolean mask indicating which data points are observed.

num_iters : int
      The number of EM iterations to run.

verbose : bool
      Whether to print progress during training.

Returns
-----
lls : Float[Tensor, "num_iters"]
      The marginal log likelihood at each iteration.

expected_states : Float[Tensor, "num_timesteps num_states"]
      The expected states,  $\Pr(z_t=k \mid r_{1:T})$  for all  $t$  and  $k$ .
"""
if mask is None:
    mask = torch.ones_like(data, dtype=bool)

lls = []
pbar = trange(num_iters) if verbose else range(num_iters)
for _ in pbar:
    expected_states, marginal_loglike = self.e_step(data, mask)
    self.m_step(data, mask, expected_states)
    lls.append(marginal_loglike)

return torch.stack(lls), expected_states

```

Now fit your model!

```

[14]: # YOUR CODE HERE
model = GaussianHMM(
    num_states=3,
    initial_probs=initial_probs,
    transition_matrix=transition_matrix
)

lls, expected_states = model.fit(r_t)

0%|          | 0/50 [00:00<?, ?it/s]

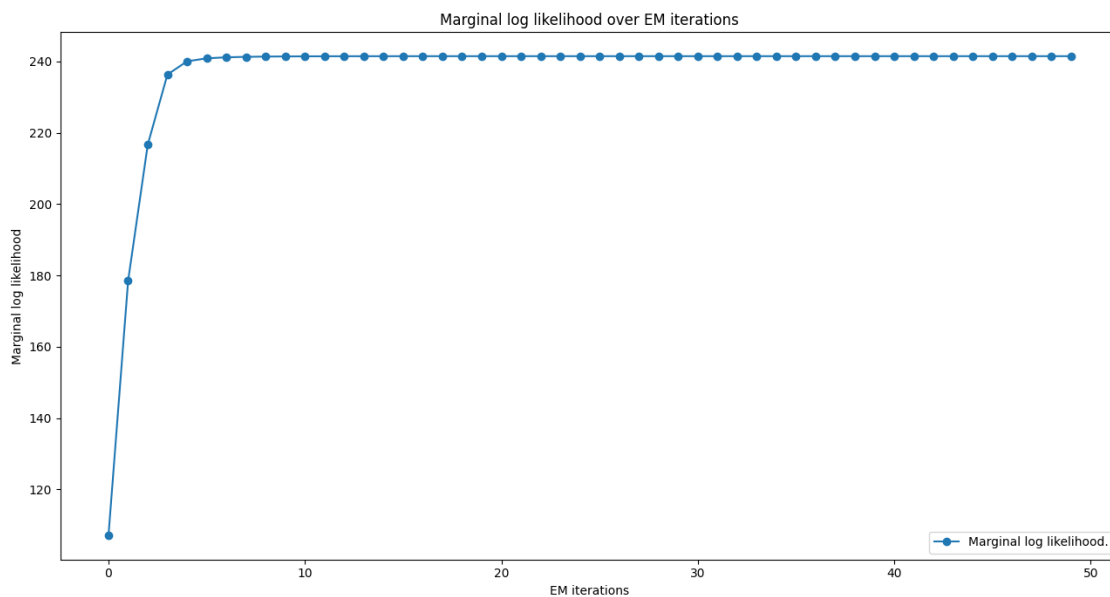
```

1.4.3 Problem 2c: Analysis

1. Plot the marginal log likelihood over EM iterations. (It should strictly increase!)

2. Print the estimate model means and variances.
3. Plot the residuals and the posterior probabilities for each of the $K = 3$ states as a function of time.

```
[23]: # YOUR CODE HERE
# 1. Plot the marginal log likelihood over EM iterations. (It should strictly
#       ↪ increase!)
plt.figure(figsize=(13, 7))
plt.plot(lls, label="Marginal log likelihood.", marker="o")
plt.xlabel("EM iterations")
plt.ylabel("Marginal log likelihood")
plt.title("Marginal log likelihood over EM iterations")
plt.legend()
plt.tight_layout()
plt.show()
```



```
[31]: # 2. Print the estimate model means and variances.
print("HMM Model means:", model.means, model.means.shape)
print("HMM Model Variances:", model.variances, model.variances.shape)
```

HMM Model means: tensor([0.0000, -0.1678, 0.2635]) torch.Size([3])

HMM Model Variances: tensor([0.0138, 0.0156, 0.0234]) torch.Size([3])

```
[38]: # 3. Plot the residuals and the posterior probabilities for each of the $K=3$
#       ↪ states as a function of time.
fig, ax = plt.subplots(4, 1, figsize=(13, 11))

ax[0].plot(dates, r_t, label="Residuals")
```

```

ax[0].set_xlabel("Date")
ax[0].set_ylabel("Residuals")
ax[0].set_title("Residuals")

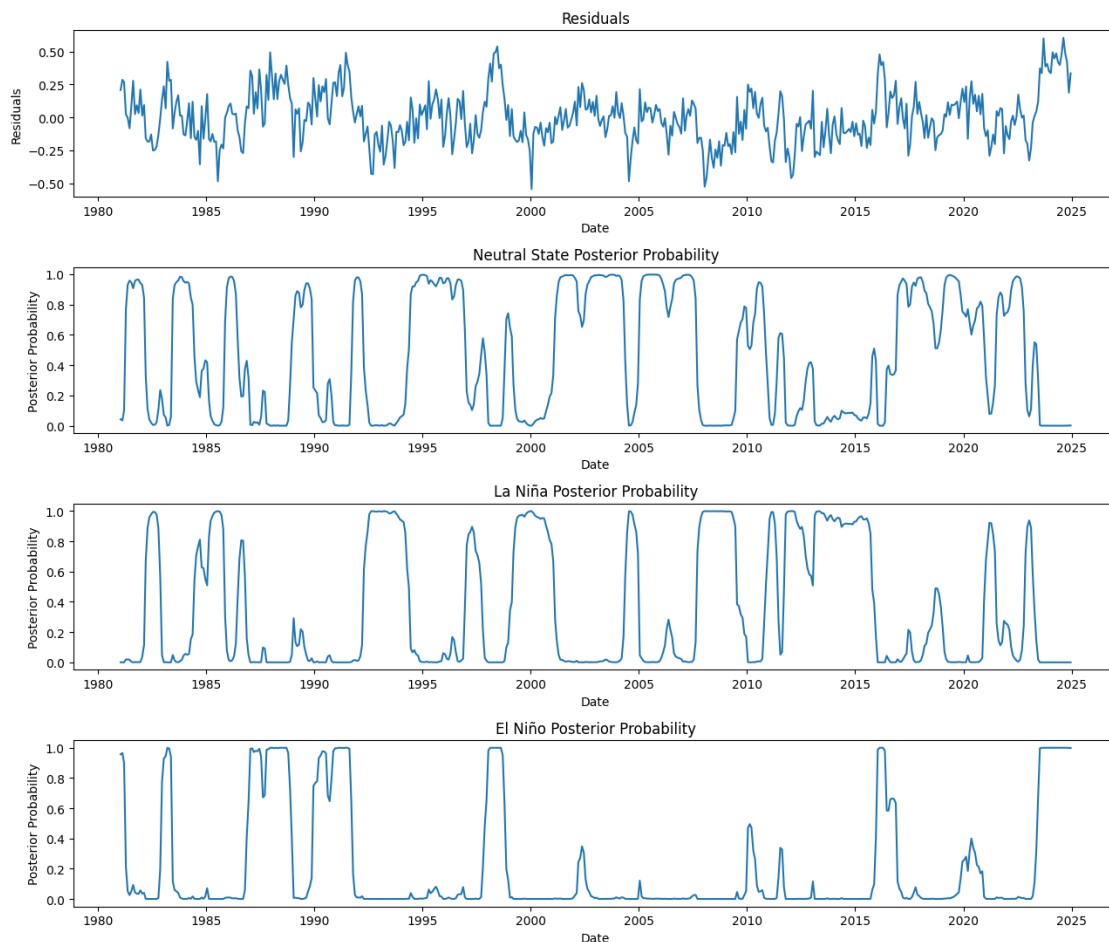
ax[1].plot(dates, expected_states[:,0], label="Neutral state")
ax[1].set_xlabel("Date")
ax[1].set_ylabel("Posterior Probability")
ax[1].set_title("Neutral State Posterior Probability")

ax[2].plot(dates, expected_states[:,1], label="La Niña")
ax[2].set_xlabel("Date")
ax[2].set_ylabel("Posterior Probability")
ax[2].set_title("La Niña Posterior Probability")

ax[3].plot(dates, expected_states[:,2], label="El Niño")
ax[3].set_xlabel("Date")
ax[3].set_ylabel("Posterior Probability")
ax[3].set_title("El Niño Posterior Probability")

plt.tight_layout()

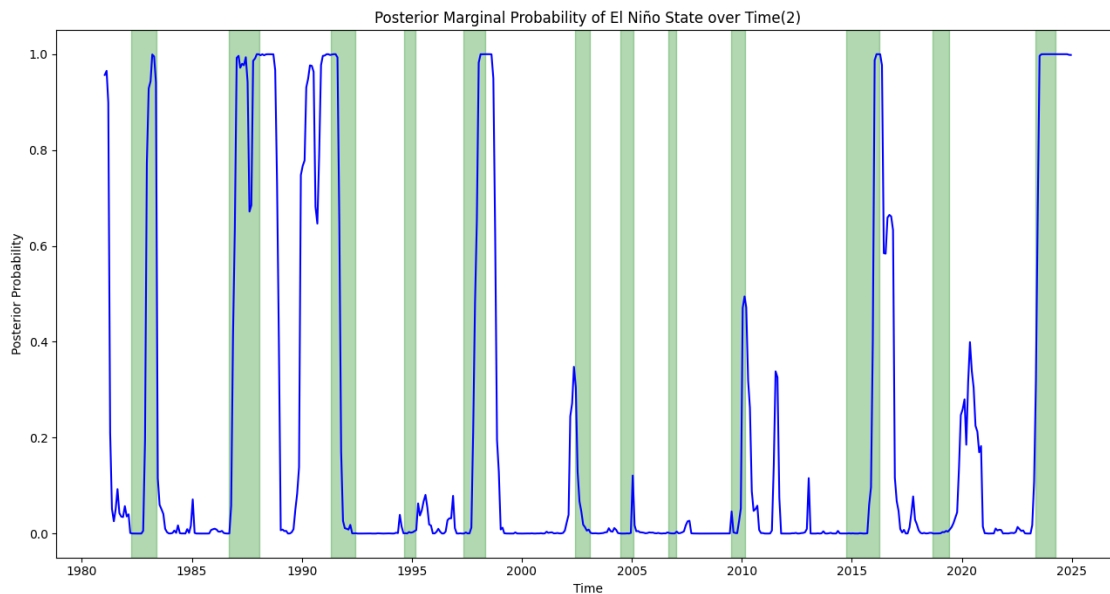
```



```
[42]: el_nino = expected_states[:, 2]
plt.figure(figsize=(13, 7))
plt.plot(dates, el_nino, label="Posterior P(El Niño state)", color='blue')

for start, end in NOAA_EL_NINO_PERIODS:
    plt.axvspan(start, end, color='green', alpha=0.3, label="NOAA El Niño_
    ↳period")

plt.xlabel("Time")
plt.ylabel("Posterior Probability")
plt.title("Posterior Marginal Probability of El Niño State over Time(2)")
plt.tight_layout()
plt.show()
```



1.4.4 Part 2d: Model Selection

One of the key hyperparameters to specify when fitting an HMM is the number of states, K . There are several strategies for choosing K . Here we will use cross-validation.

1. Hold out the observations for years 1983, 1988, 1992, 1999, 2004, 2010, 2015, 2020, and 2022 using the mask vector provided below. The mask is a vector $m_{1:T}$ that is zero for all time points in those years and one otherwise.
2. Fit a Gaussian HMM with $K = 1, 2, \dots, 10$ states using your code above, but now provide the mask so that the model is fit based on only the training data. Remember the EM can get stuck in local optima, so fit the model several times from random initializations and keep the

best model fit.

3. For each value of K , evaluate the **log predictive likelihood** of the held-out data under the best fitting model. You can do this by computing

$$\log p(\mathbf{r}_{\text{test}} \mid \mathbf{r}_{\text{train}}; \hat{\theta}) = \log p(\mathbf{r}; \hat{\theta}) - \log p(\mathbf{r}_{\text{train}}; \hat{\theta})$$

where $\mathbf{r}_{\text{train}} = \{r_t : m_t = 1\}$ denotes the subset of training data, $\mathbf{r}_{\text{test}} = \{r_t : m_t = 0\}$ denotes the held-out test data, and $\hat{\theta}$ denotes the parameters estimated with EM from the best random initialization. You can compute these marginal likelihoods by calling the `e_step` function with the appropriate mask.

4. Discuss your findings. How many discrete states would you recommend? Visualize the states of the best-fitting model with the chosen value of K . Can you interpret the states?

```
[43]: # We provide the followign mask
heldout_years = torch.tensor([1983, 1988, 1992, 1999, 2004, 2010, 2015, 2020,
↪2022])
mask = torch.tensor([date.astype(datetime).year not in heldout_years for date
↪in dates], dtype=bool)
```

```
[44]: # YOUR CODE HERE
# I didn't have enough time...
# credits: https://github.com/dlvh/stats305b/blob/main/assignments/hw3/hw3.ipynb

# Storing Results. lls aka log likelihoods
predictive_lls = []
best_models = {}

# for K in 1, 2, ..., 10:
for K in range(1, 11):
    best_train_ll = float('-inf')
    best_model = None
    # Fit the model several times from random initialization
    # This is item 2: keeping the best model fit
    for run in range(5):
        initial_probs = torch.ones(K) / K
        # constructing the transition matrix for this K
        alpha = 11/12
        transition_matrix = torch.full((K, K), (1 - alpha) / (K - 1) if K > 1 else
↪0.0)
        if K > 1:
            transition_matrix.fill_diagonal_(alpha)
        else:
            transition_matrix = torch.ones(1, 1)
        # initialize HMM
        model = GaussianHMM(num_states=K, initial_probs=initial_probs,
↪transition_matrix=transition_matrix)
        # fit the model (to compare the training lls)
```

```

    lls, _ = model.fit(r_t, mask=mask, num_iters=50, verbose=False)
    train_ll = lls[-1]
    if train_ll > best_train_ll:
        best_train_ll = train_ll
        best_model = model
    # now, the best_model is the model with the best random initialization
    # want to get the log predictive likelihood
    # This is item 3: getting the ll from e_step
    _, full_ll = best_model.e_step(r_t, mask=torch.ones_like(mask, dtype=bool)) #_
    ↪ NO MASK
    _, train_ll = best_model.e_step(r_t, mask=mask)
    pred_ll = full_ll - train_ll
    # store the results
    predictive_lls.append(pred_ll.item())
    best_models[K] = best_model
    # printing results
    print(f"K={K}: Train LL = {train_ll.item():.2f}, Full LL = {full_ll.item():.
    ↪ 2f}, Predictive LL = {pred_ll.item():.2f}")

```

```

K=1: Train LL = 72.84, Full LL = 102.77, Predictive LL = 29.93
K=2: Train LL = 121.67, Full LL = 158.26, Predictive LL = 36.60
K=3: Train LL = 117.92, Full LL = 151.01, Predictive LL = 33.09
K=4: Train LL = 203.55, Full LL = 255.57, Predictive LL = 52.02
K=5: Train LL = 211.41, Full LL = 262.86, Predictive LL = 51.45
K=6: Train LL = 204.07, Full LL = 251.07, Predictive LL = 47.00
K=7: Train LL = 212.33, Full LL = 259.36, Predictive LL = 47.02
K=8: Train LL = 205.95, Full LL = 252.08, Predictive LL = 46.13
K=9: Train LL = 210.11, Full LL = 257.44, Predictive LL = 47.32
K=10: Train LL = 202.15, Full LL = 241.30, Predictive LL = 39.15

```

```

[45]: # Printing out Findings
best_K = max(range(1, 11), key=lambda k: predictive_lls[k - 1])
print("\nBest number of states K:", best_K)

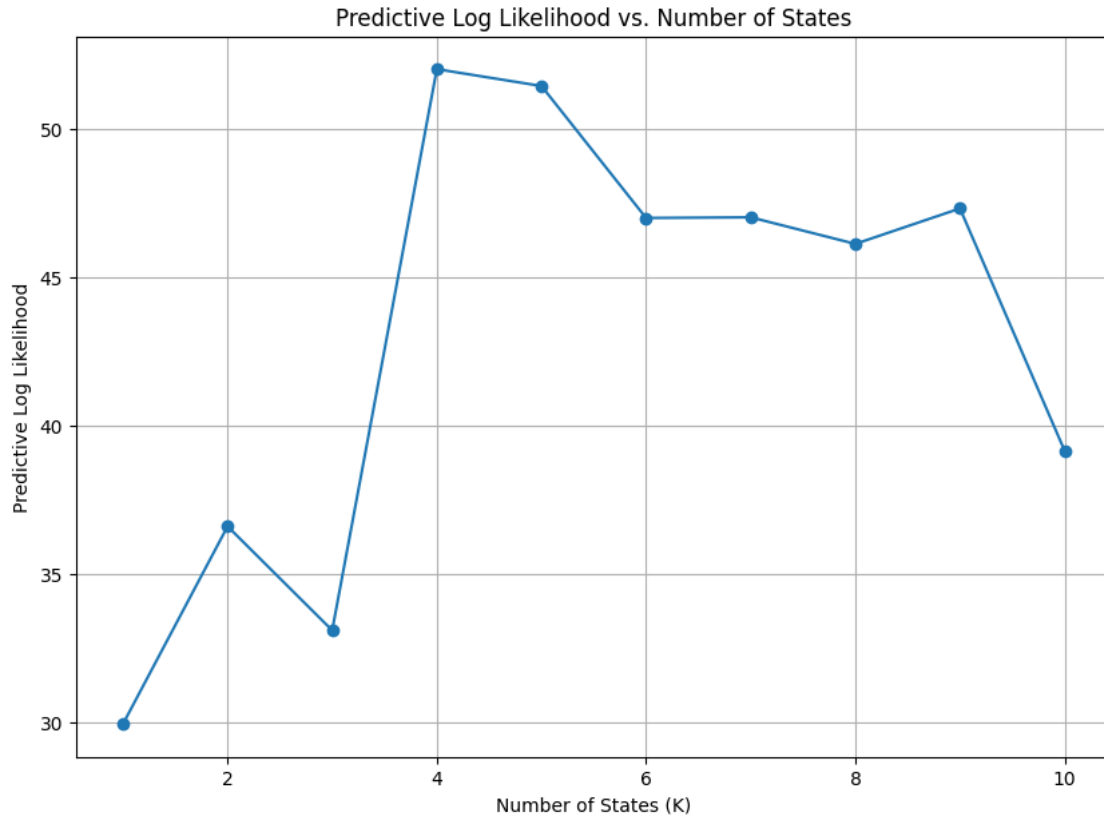
```

Best number of states K: 4

```

[47]: # Item 4: plotting the predictive log likelihood vs K
plt.figure(figsize=(10, 7))
plt.plot(range(1, 11), predictive_lls, marker='o', linestyle='--')
plt.xlabel("Number of States (K)")
plt.ylabel("Predictive Log Likelihood")
plt.title("Predictive Log Likelihood vs. Number of States")
plt.grid(True)
plt.show()

```



```
[ ]: # Item 4: Visualize the states with the best fitting model
# Residuals Time Series and State Assignments
# Setup to K = 4
best_model = best_models[best_K]
expected_states_best, _ = best_model.e_step(r_t, mask=torch.ones_like(mask, dtype=bool))
state_sequence = torch.argmax(expected_states_best, dim=1)

for k in range(best_K):
    plt.figure(figsize=(13, 7))
    plt.plot(dates, r_t, label="Observed Data", color='gray', alpha=0.5)

    # Get indices where the most likely state equals k
    indices = (state_sequence == k).nonzero(as_tuple=True)[0]

    # Plot only the data points assigned to state k
    plt.scatter([dates[i] for i in indices], r_t[indices],
                color=f"C{k}", label=f"State {k} assignments")

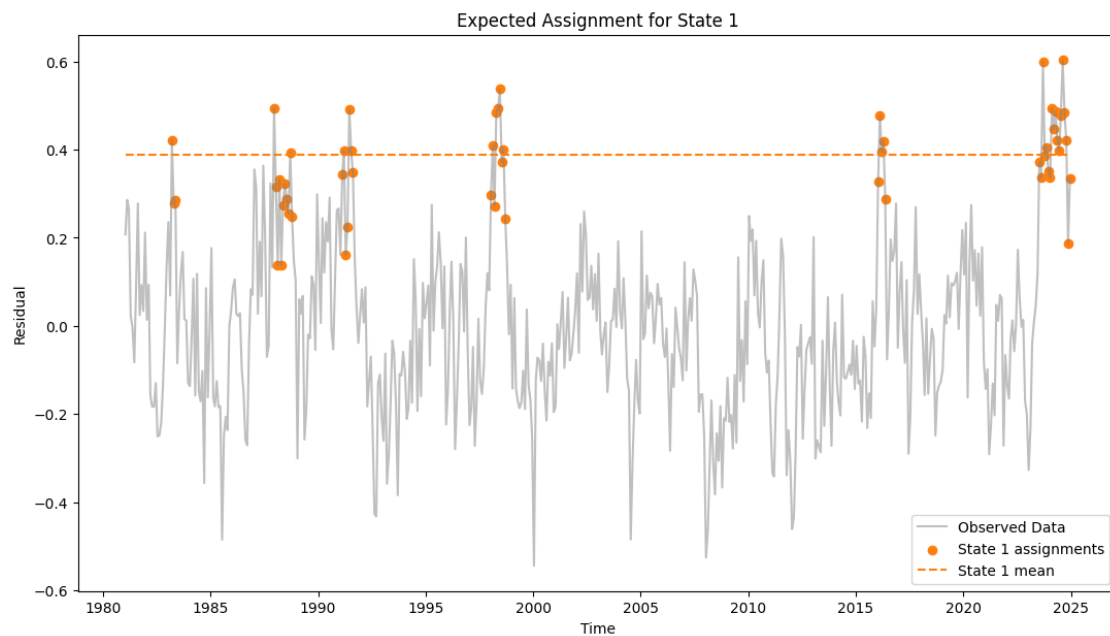
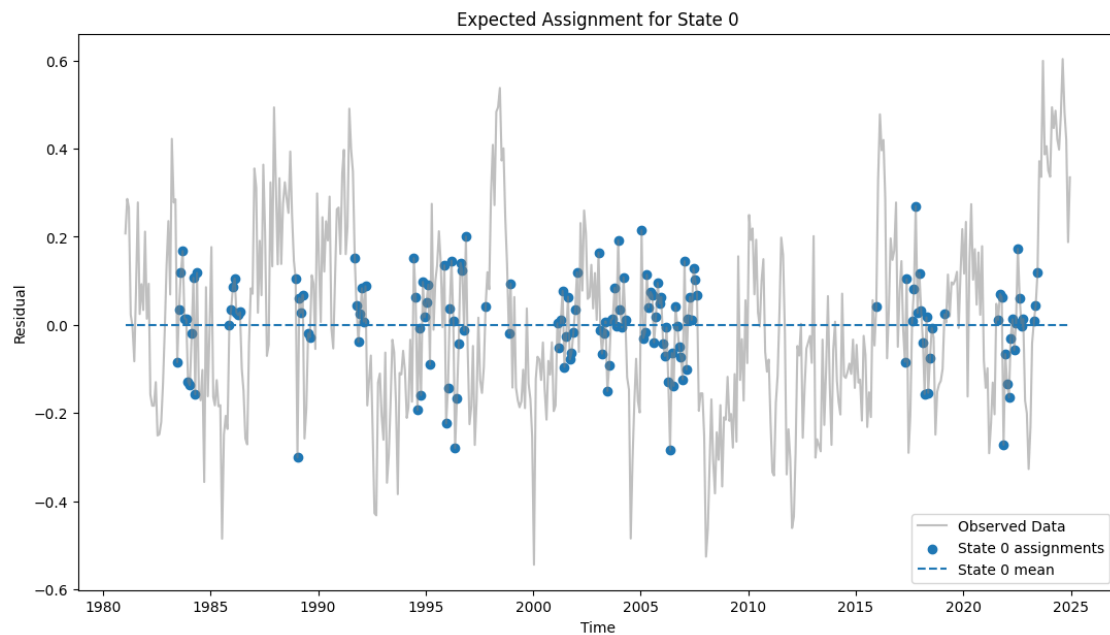
    # Overlay the state mean as a horizontal dashed line
    plt.hlines(best_model.means[k].item(), dates[0], dates[-1],
```

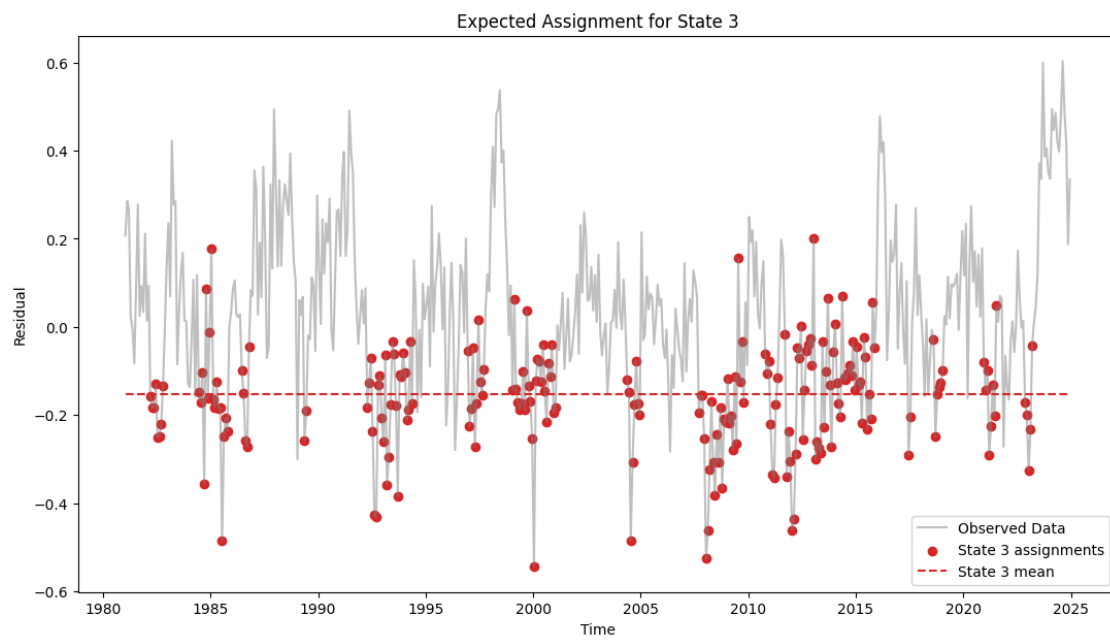
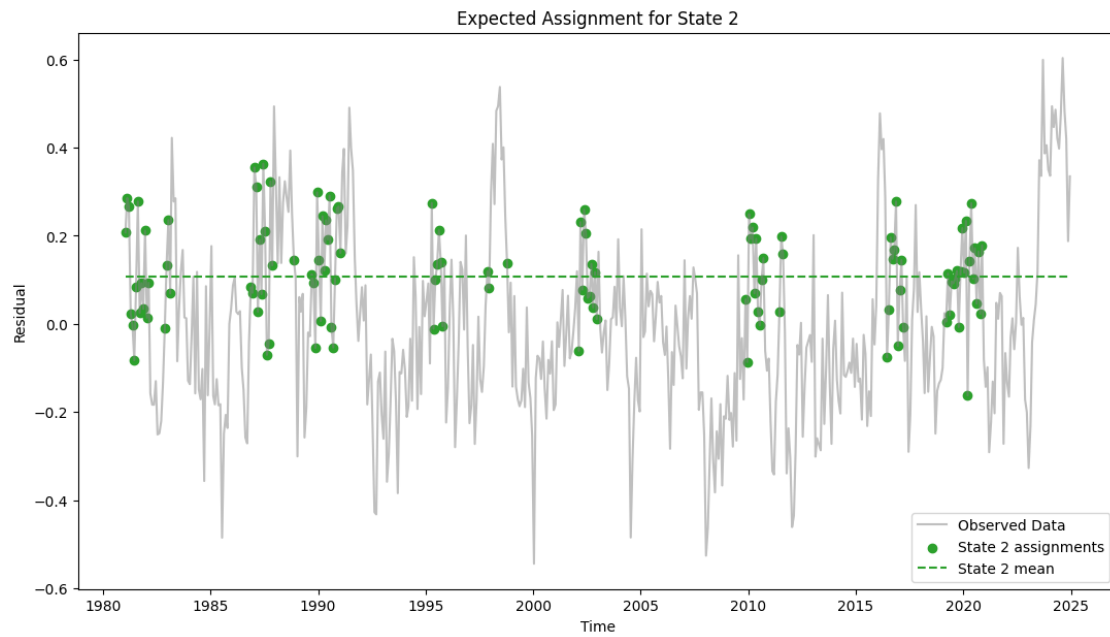
```

linestyles="--", colors=f"C{k}", label=f"State {k} mean")

plt.xlabel("Time")
plt.ylabel("Residual")
plt.title(f"Expected Assignment for State {k}")
plt.legend()
plt.show()

```





```
[52]: # Item 4: Visualize the states with the best fitting model
      # Posterior Probabilities
      fig, ax = plt.subplots(4, 1, figsize=(13, 11))
```

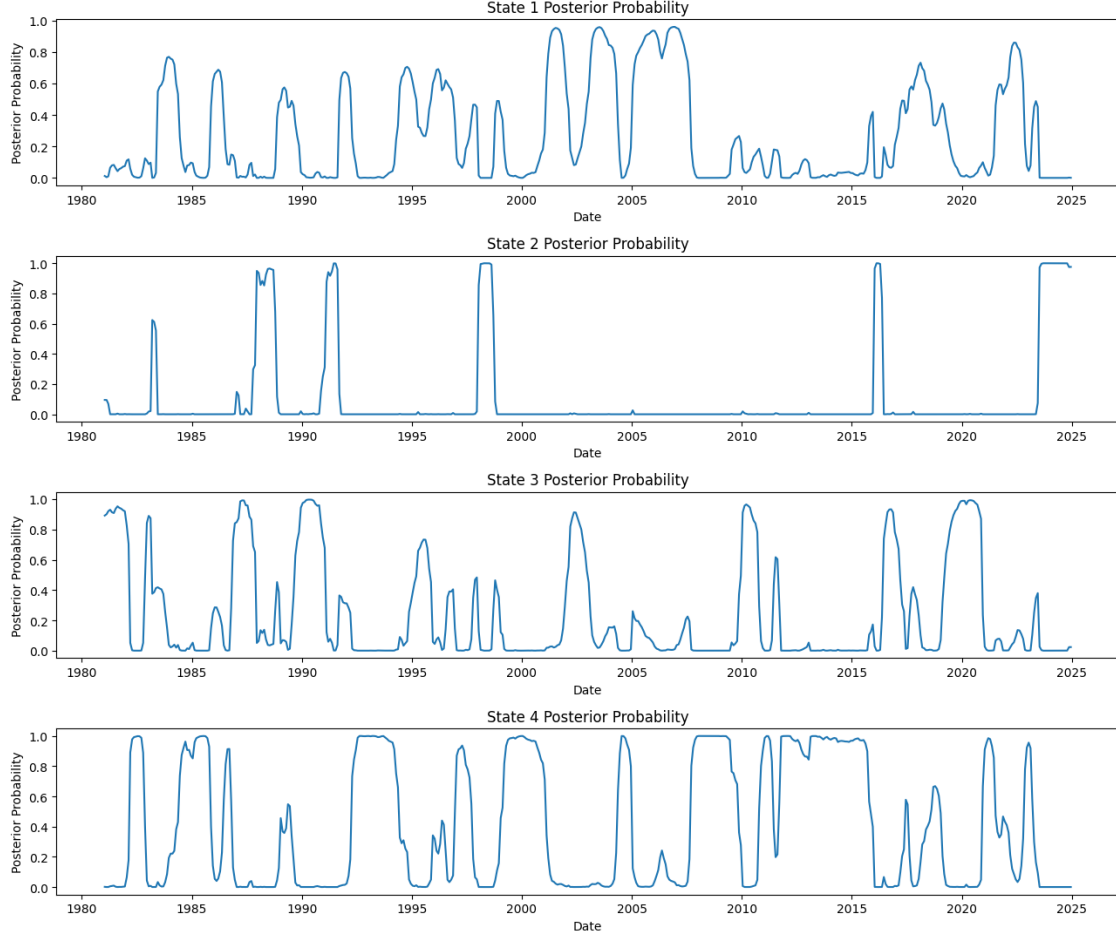
```
ax[0].plot(dates, expected_states_best[:,0], label='State 1 (Mean Zero)')
ax[0].set_xlabel('Date')
ax[0].set_ylabel('Posterior Probability')
ax[0].set_title('State 1 Posterior Probability')

ax[1].plot(dates, expected_states_best[:,1], label='State 2')
ax[1].set_xlabel('Date')
ax[1].set_ylabel('Posterior Probability')
ax[1].set_title('State 2 Posterior Probability')

ax[2].plot(dates, expected_states_best[:,2], label='State 3')
ax[2].set_xlabel('Date')
ax[2].set_ylabel('Posterior Probability')
ax[2].set_title('State 3 Posterior Probability')

ax[3].plot(dates, expected_states_best[:,3], label='State 4')
ax[3].set_xlabel('Date')
ax[3].set_ylabel('Posterior Probability')
ax[3].set_title('State 4 Posterior Probability')

plt.tight_layout()
```



Discuss: The interpretation of the four hidden states is not entirely straightforward, as they do not exhibit clear geographical patterns. Nevertheless, a closer examination of the fitted HMM suggests that states 2 and 3 together resemble characteristics commonly associated with El Niño conditions, whereas state 4 appears more consistent with La Niña conditions.

This interpretation is also broadly consistent with the residual structure of our linear regression baseline. During El Niño periods, global temperatures tend to be anomalously warm, which corresponds to the relatively positive residuals observed in our model. Conversely, during La Niña periods, global temperatures tend to be anomalously cool, corresponding to more negative residuals.

It is worth noting that hidden states identified by the EM algorithm are optimized to maximize the data likelihood rather than to correspond directly to physically meaningful climate regimes. As a result, the algorithm may split what is effectively a single underlying climate state into multiple hidden states in order to achieve a better statistical fit. This may help explain why the model selection procedure favors ($K = 4$), even though the resulting states are not all easily interpretable from a climatological perspective.

1.5 Part 3: Model Criticism and Revision

You’ve completed another lap of Box’s Loop! We worked with real climate data, developed an HMM to infer El Niño and La Niña periods, and implemented the forward-backward and EM algorithms for state inference and parameter estimation. We even used cross-validation to select the number of states for our model. Well done!

Of course, Box’s Loop wouldn’t be a loop if we stopped there. In this part, let’s think about ways we could improve these analyses.

1.5.1 Problem 3a: Change Point Analysis

Our analyses were based on residuals from a baseline linear model that assumed a constant rate of warming over the past 40+ years. However, it’s possible that the rate has changed over time as greenhouse gas emissions have varied. There are many ways to estimate nonlinear trends — one way is to use an HMM.

Recall that in our baseline model, the coefficients were $\beta = (\beta_1, \dots, \beta_p)$ where β_1 represented the rate of warming. Suppose the rate jumped from $\beta_1^{(1)}$ to a new value, $\beta_1^{(2)}$, at some unknown time between 1981 and 2024. Describe how you could use an HMM to infer the time of the change point. Define the model mathematically and specify any relevant constraints on the parameters. You should also account for monthly effects, as in the baseline model.

You do not have to fit the model, only specify it and justify your reasoning.

Answer:

Let $z_t \in \{0, 1\}$ denote the latent regime at time t , where $z_t = 0$ means the process is still in the pre-change regime and $z_t = 1$ means the post-change regime has begun. We assume a one-way transition:

$$P(z_{t+1} = 1 \mid z_t = 0) = \eta, \quad P(z_{t+1} = 0 \mid z_t = 0) = 1 - \eta, \quad P(z_{t+1} = 1 \mid z_t = 1) = 1,$$

so state 1 is absorbing and the change point can occur at most once. We also set $P(z_1 = 0) = 1$. If τ denotes the change point time, then $\tau = \min t : z_t = 1$.

Conditional on z_t , we model the temperature with a Gaussian regression that includes the same monthly effects as the baseline model:

$$y_t \mid z_t = k \sim \mathcal{N}(\beta_0 + \beta_1^{(k)}t + \beta_m^\top \mathbf{x}_t^m, \sigma^2), \quad k \in \{0, 1\},$$

where $\mathbf{x}_t^m$ is a month indicator vector. To ensure identifiability, one should either omit one month indicator or impose a constraint such as $\sum_m \beta_m = 0$.

This HMM encodes a single persistent change in the warming rate: the slope before the change is $\beta_1^{(0)}$, and after the change it is $\beta_1^{(1)}$. After fitting the model, the posterior over the change point can be recovered from the smoothed transition probabilities,

$$P(\tau = t \mid y_{1:T}) = P(z_{t-1} = 0, z_t = 1 \mid y_{1:T}),$$

and the most likely change point is $\hat{\tau} = \arg \max_t P(\tau = t \mid y_{1:T})$.

1.5.2 Problem 3b: Spatiotemporal Analysis

The NOAA data has rich spatiotemporal structure, but we ignored the spatial component and just modeled the global mean temperature! It's possible that some parts of the globe are warming faster than others. Some parts might even be cooling over time. Likewise, anomalies like El Niño could affect the temperature in some regions more than others.

Describe, mathematically, how you would model the full spatiotemporal dataset. What challenges can you anticipate, and how would you address them?

You do not have to fit the model, only specify it and justify your reasoning.

Answer:

Let $y_t(s)$ denote the temperature anomaly at location s and time t , or equivalently let $y_t \in \mathbb{R}^N$ be the vector of temperatures on an N -grid at time t . A natural spatiotemporal model is a low-rank factor model with temporal dynamics:

$$y_t = \mu + Bf_t + \epsilon_t,$$

where $\mu \in \mathbb{R}^N$ is the spatial mean field, $B \in \mathbb{R}^{N \times r}$ contains spatial basis functions or loadings, $f_t \in \mathbb{R}^r$ are latent temporal factors, and $\epsilon_t \sim \mathcal{N}(0, \Sigma_\epsilon)$. The latent factors may follow a state-space or HMM dynamics, for example

$$f_t = Af_{t-1} + u_t, \quad u_t \sim \mathcal{N}(0, Q),$$

or, to capture regime changes such as El Niño / La Niña, one may introduce a discrete latent state z_t and let

$$f_t \mid z_t = k \sim \mathcal{N}(m_k, Q_k), \quad z_t \sim \text{Markov}.$$

This model allows different regions to respond differently over time while still borrowing strength across the spatial field.

The main challenges are high dimensionality, strong spatial dependence, missing observations, and nonstationarity. These can be addressed by using a low-rank basis representation or sparse Gaussian-process approximations to reduce computational cost, imposing identifiability constraints on B , and modeling time dynamics in the latent factors rather than fitting each grid cell independently. Compared with a raw PCA preprocessing step, this formulation keeps the spatial structure inside the probabilistic model and is therefore more principled.

1.6 Submission Instructions

Formatting: check that your code does not exceed 80 characters in line width. You can set *Tools* → *Settings* → *Editor* → *Vertical ruler column* to 80 to see when you've exceeded the limit.

Converting to PDF The simplest way to convert to PDF is to use the “Print to PDF” option in your browser. Just make sure that your code and plots aren't cut off, as it may not wrap lines.

Alternatively You can download your notebook in .ipynb format and use the following commands to convert it to PDF. Then run the following command to convert to a PDF:

```
jupyter nbconvert --to pdf <yourlastname>_hw<number>.ipynb
```

(Note that for the above code to work, you need to rename your file `<yourlastname>_hw<number>.ipynb`)

Installing nbconvert:

If you're using Anaconda for package management,

```
conda install -c anaconda nbconvert
```

Upload your .pdf file to Gradescope. Please tag your questions correctly! I.e., for each question, all of and only the relevant sections are tagged.

Please post on Ed or come to OH if there are any other problems submitting the HW.